

1. Technical Debt Diagnosis

1.1 Identify the hidden-technical-debt categories

(a) **Entanglement**

- The weights are fit together, so the coupling is in the model, not the code. Nothing links the two features.
- Rounding delivery time threw away signal. The optimiser made it up elsewhere, and favourite-restaurants ended up standing in for it. Nothing broke, the recommendations just got worse.

(b) **Undeclared Consumers**

- Marketing started reading the output table without telling anyone, so the ML team's dependency graph is wrong.
- A schema tweak, a rescaling, or a routine retrain all look safe from the inside, and any of them can break a campaign the team doesn't know about.

(c) **Configuration & Glue-Code Debt**

- The 14 chained scripts are the pipeline jungle. Extraction, transformation and model calls are taped together with no orchestration layer, and the hyperparameters sit inside the scripts.
- The manual steps were never written down, so nobody can reproduce the setup off the author's machine.

1.2 Mitigation for (c)

Tool: MLflow Projects - an `MLproject` file with declared entry points, a `python_env.yaml`, run via `mlflow run`.

- Swap the 14 scripts for one `MLproject` file with named entry points (`main`, `evaluate`), typed parameters and defaults, and a `python_env.yaml` pinning the interpreter and library versions.
- **Configuration Debt:** hyperparameters leave the shell scripts and become declared parameters with one default, overridden at the command line via `mlflow run . -P n_estimators=300`.
- **Glue-Code Debt:** the chain becomes one documented command that runs locally or from a Git URL.
- Each `mlflow run` opens a Tracking run, logging params, metrics and the `git_commit` tag. That is the run pinned.